Folder · one page inside one folder it owns

1 You see what your folder holds, and what has gone out of date

The tab: the first tab on the rail shows what this page's own folder holds, read fresh every time you open it.

```
   QPf1-folder/                           Folder tab (first on the rail)
  +-- bibex/  0 files  ---->    bibex/   rebuilt for you . empty . [x] fresh
  +-- draw/   1 file   ---->    draw/    age shown . never flagged
  +-- skill/  2 files  ---->    skill/   age shown . never flagged
  +-- slide/  1 file   ---->    slide/    STALE .  regenerate in the  tab
  +-- QPf1-folder.md    the clock every REBUILT folder is checked against
                               [ ] not present: chat . latex . word . display . meeting
```

The page's own folder is listed right on the page, with a warning on any part that has fallen behind the page.

First, whether "no deck" means a deck was never built, or was built and never opened. Second, whether a built file still matches the words it was made from. The tab answers both.

A row reads STALE only if a machine rebuilds that folder: latex, word, bibex, slide, and display. It also has to hold files, with its newest file older than the page's `.md`. An empty one is never flagged, so this page's own `bibex/` holds nothing and reads [x] fresh. A folder you write yourself shows an age and never a warning. A drawing older than the words is normal, not a problem.

`GET /_board/folderstat` walks the folder each time you open the tab, and it saves nothing. A saved status starts going out of date the moment it is written, and a stale report about staleness is worse than none. Two layers keep it fresh. The server sends `no-store`, and the page shell reloads the panel even when the address has not changed. That second layer matters, because this view has one address per page, so "same address, skip it" would hand you yesterday's list.

A STALE row can be fixed on the spot when a machine wrote its files. The latex, word, bibex, and display rows carry rebuild. One click runs that folder's own rebuild, then reads the status again. The tab header does the same for all of them at once: rebuild stale (n). It works through every fixable row one after another, and it shows only while a machine-written row is out of date. Slide sends you to the tab's instead, because a deck someone wrote must never be rebuilt just from looking around.

2 Click a row and the real files open

Click to look inside: a row opens its own folder shape in place, and every file link opens the real file.

```
    pagex/   6 files           click the row ...
    pagex/   6 files           ... and it unfolds as a TREE:
      QPf11-pagex.md              the files this folder OWNS come first
      QPf11-pagex-view.html
       QPs1-overall/   1 file    then one branch per folder under it
          QPs1-overall.md        a  marks a symlink, target on hover
```

Every row opens into a tree of real files, and each link takes you to the file itself.

The first build listed every path flat and in alphabetical order. So `pagex/` read as six rows with nothing joining them, its own list file and view file stuck between four borrowed pages (JL 260816: "   "). The shape of a folder is the point. A level's own files come first, then the folders under it. Each branch shows its own file count, so one look is still worth something. The same change made `display/` readable, and its parts sit three folders deep. A   marks a row whose file is a symlink, and hovering shows the full target. The row reports the file the link points at, so without the   a borrowed page would look like a copy (JL 260816: "are they copied or are they the symlink?").

The links are plain web addresses under the board tree. The browser shows a pdf or an html page straight away and offers the rest as a download. Nothing is copied, and nothing is wrapped. So the tab is both a meter and a door. The same walk that counts the folder also lists it, so what you click is exactly what was counted.